Engineering Practices for Efficient and Reliable Small Language Model Inference

A Reproducible Case Study of Quantization and Inference Performance

Seyedmahdi Masoumi
Independent Research Project

October 2026

Abstract

Efficient deployment of small language models requires balancing memory requirements, inference performance, and output reliability. Although low-bit quantization can reduce model memory consumption, its effects on generation speed and response behavior depend on the selected hardware and inference implementation.

This study presents an empirical case study comparing FP16 and 4-bit NormalFloat (NF4) inference for Qwen2.5-0.5B-Instruct on an NVIDIA Tesla T4 GPU. The evaluation examines PyTorch GPU memory allocation during model loading, generation throughput and latency across 20 prompts with two repetitions per configuration, and exploratory response quality and instruction compliance using the same 20 base prompts with an additional response-length instruction and different generation settings.

The NF4 configuration reduced measured model-loading GPU memory allocation by approximately 52.2% compared with FP16. However, FP16 achieved higher mean generation throughput (29.75 versus 19.94 tokens/s) and lower mean generation latency (2.15 versus 3.25 seconds).

An exploratory AI-assisted quality assessment produced mean scores of 3.1/4 for FP16 and 3.0/4 for NF4. The mean paired difference was 0.10 points, with a bootstrap 95% confidence interval of [-0.35, 0.55]. Compliance with an explicit 100-word response limit was 50% for FP16 and 85% for NF4.

These findings illustrate that reduced model-loading memory allocation does not necessarily translate into faster inference or uniformly improved output behavior. The study highlights the importance of multi-metric evaluation when selecting resource-aware inference configurations. Its conclusions are limited by the single-model, single-GPU setup, small prompt collection, and exploratory quality assessment. Energy consumption and digital sovereignty were not directly evaluated.

1 Introduction

Generative artificial intelligence (AI) has created new opportunities for integrating language-based capabilities into software systems. However, deploying language models involves practical engineering challenges, including memory requirements, inference latency, computational cost, and the reliability of generated outputs. These challenges are particularly relevant when AI

systems must operate under limited hardware resources or within infrastructure controlled by the deploying organization.

Small language models (SLMs) offer a potential alternative to larger models for applications where computational resources, deployment costs, or infrastructure constraints are important. Their smaller parameter counts can make local deployment more accessible, although practical efficiency depends on model architecture, numerical precision, hardware characteristics, and inference implementation.

Model quantization is one technique for reducing the memory required to represent neural network weights. By storing weights at lower numerical precision, quantization can reduce memory requirements and potentially enable deployment on more constrained hardware. Nevertheless, lower numerical precision does not guarantee faster inference. Runtime performance also depends on factors such as quantization and dequantization overhead, computational kernels, memory access patterns, and hardware support. Furthermore, changes in numerical representation may affect generated outputs, making response reliability an important consideration alongside resource efficiency.

These considerations motivate an empirical approach to inference optimization. Rather than assuming that a more compact model representation is universally preferable, deployment decisions should be informed by measurements of memory usage, generation speed, latency, and output behavior under clearly specified experimental conditions.

This study investigates these trade-offs through a case study of Qwen2.5-0.5B-Instruct, comparing half-precision floating-point (FP16) inference with 4-bit NormalFloat (NF4) weight quantization on an NVIDIA Tesla T4 GPU. The evaluation includes measurements of PyTorch GPU memory allocation during model loading, generation throughput and latency, and exploratory assessment of response quality and instruction compliance.

The main contribution of this work is a documented, multi-metric experimental case study that illustrates the practical trade-offs between memory efficiency, inference performance, and generated response behavior for the selected model and hardware configuration. The findings are intended to inform resource-aware inference engineering, rather than establish universal performance claims about quantization.

Although resource-efficient inference is relevant to sustainable AI deployment, this study does not directly measure energy consumption or carbon emissions. Similarly, while local model deployment can be relevant to digital sovereignty, the experiments do not directly evaluate data governance, organizational autonomy, or dependence on external AI service providers. These broader implications are therefore treated as motivation and potential directions for future research rather than demonstrated outcomes.

2 Background and Related Work

2.1 Efficient Language Model Inference

Deploying language models involves trade-offs between computational requirements, memory consumption, inference performance, and output reliability. These trade-offs become particularly important when models are deployed on hardware with limited memory capacity or under

application-specific latency constraints.

Reducing numerical precision is one approach to decreasing model memory requirements. However, the performance effects of lower-precision inference depend on both the quantization method and its hardware-specific implementation.

Dettmers et al. [1] introduced LLM.int8(), a mixed-precision approach designed to enable lower-memory inference while preserving the performance of large transformer models.

Their work demonstrates the importance of numerical representation in practical language model deployment. Nevertheless, the behavior of an 8-bit method cannot be assumed to characterize all lower-precision inference configurations.

2.2 Four-Bit NormalFloat Quantization

Four-bit NormalFloat (NF4) quantization was introduced as part of the QLoRA approach by Dettmers et al. [2].

QLoRA combines a frozen quantized base model with trainable low-rank adapters to reduce the memory requirements of fine-tuning.

NF4 provides a four-bit representation designed for normally distributed neural network weights.

Although QLoRA primarily addresses memory-efficient fine-tuning, NF4 weight quantization can also be used in inference configurations through supported implementations.

Importantly, reducing the number of bits used to represent weights does not necessarily reduce generation latency.

Inference performance may depend on quantization and dequantization operations, kernel implementations, and the capabilities of the underlying hardware.

Consequently, the performance of NF4 inference should be evaluated empirically rather than inferred directly from its weight-storage requirements.

2.3 Small Language Models and Qwen2.5

Small language models offer a potentially practical option for applications where hardware resources or deployment costs are constrained.

The Qwen2.5 model family is described in the Qwen2.5 Technical Report [3].

This family includes models of different parameter scales, providing options for investigating language model behavior under different resource constraints.

The present study uses Qwen2.5-0.5B-Instruct as a small-scale experimental subject for comparing FP16 and NF4 inference configurations.

The objective is not to benchmark the Qwen2.5 family comprehensively, but to examine the measurable trade-offs associated with two numerical representations for one selected model.

2.4 Positioning of the Present Study

Prior research has established quantization as an important technique for reducing language model memory requirements.

However, the operational effects of quantization depend on the specific model, inference implementation, hardware, and evaluation workload.

This study contributes a documented experimental case study that evaluates several deployment-relevant dimensions within a single configuration: model-loading GPU memory allocation, generation throughput, latency, exploratory response quality, and instruction compliance.

Rather than proposing a new quantization algorithm, the study investigates the practical engineering consequences of using an existing quantization method.

The results are intended to support evidence-based reasoning about resource-aware inference deployment and to motivate broader evaluations of sustainable and independently managed AI systems.

3 Research Questions

The objective of this study is to investigate the practical trade-offs associated with 4-bit NF4 weight quantization compared with FP16 inference for a small language model under a fixed hardware configuration.

The investigation is guided by four research questions:

RQ1 – Memory Efficiency: How does 4-bit NF4 quantization affect PyTorch GPU memory allocation during model loading compared with FP16?

RQ2 – Inference Performance: How do FP16 and 4-bit NF4 configurations compare in terms of generation throughput and latency across a fixed set of prompts?

RQ3 – Response Quality: What differences in response quality are observed between FP16 and 4-bit NF4 configurations under deterministic decoding, based on an exploratory AI-assisted evaluation?

RQ4 – Instruction Compliance: How do the two configurations compare in their ability to satisfy an explicit 100-word response limit?

These questions are evaluated using the same model architecture and GPU type, with controlled generation settings within each experimental task.

The study focuses on observed behavior under the selected experimental conditions. It does not seek to establish general superiority of either numerical representation across different models, hardware platforms, or deployment scenarios.

4 Experimental Methodology

4.1 Experimental Environment

All experiments were conducted in Google Colab using an NVIDIA Tesla T4 GPU with approximately 14.56 GiB of total device memory. The software environment included Python 3.13.15, PyTorch 2.11.0+cu130, Transformers 4.57.6, Accelerate 1.15.0, and BitsAndBytes 0.50.2.

The selected model was `Qwen/Qwen2.5-0.5B-Instruct`, an instruction-tuned small language model. The same model and tokenizer were used for both inference configurations.

4.2 Inference Configurations

Two numerical representations were evaluated:

- **FP16:** Model weights were loaded using half-precision floating-point representation.
- **4-bit NF4:** Model weights were loaded using BitsAndBytes 4-bit NormalFloat quantization, with FP16 computation and double quantization disabled.

Both configurations were deployed on the same GPU type using Hugging Face Transformers. The models were placed in evaluation mode, and generation was performed using `torch.inference_mode()`.

Greedy decoding was used throughout the experiments by setting `do_sample=False`.

4.3 GPU Memory Measurement

GPU memory allocation was measured separately for each configuration during model loading.

Before loading each model, Python garbage collection and PyTorch CUDA cache cleanup were performed. GPU synchronization was also applied.

The allocated GPU memory was recorded immediately before and after loading the model. The reported measurement was calculated as:

$$M_{\text{load}} = \frac{M_{\text{after}} - M_{\text{before}}}{1024^3}, \tag{1}$$

where memory allocation is measured in bytes, and the resulting value is expressed in GiB.

The measurement represents the change in PyTorch-reported GPU memory allocation during model loading. It does not represent total device memory usage, peak inference memory, or energy consumption.

The FP16 and NF4 models were measured sequentially, with cleanup between runs.

4.4 Inference Performance Benchmark

A fixed set of 20 English-language prompts covering introductory AI and software engineering topics was used to evaluate generation performance.

Each prompt was evaluated twice for each configuration, producing 40 timed generations per configuration.

The input prompts were formatted using the model's chat template. Each generation used a maximum of 64 new tokens and deterministic greedy decoding.

Before the timed benchmark for each configuration, a warm-up generation of up to 32 tokens was executed using a separate greeting prompt.

Generation time was measured using `time.perf_counter()`, with `torch.cuda.synchronize()` immediately before and after each timed generation.

Latency was defined as the elapsed time for the generation call:

$$L_i = t_{\text{end},i} - t_{\text{start},i}. \tag{2}$$

Throughput was calculated as:

$$T_i = \frac{N_i}{L_i}, \tag{3}$$

where N_i denotes the number of generated tokens and L_i is the measured generation latency in seconds.

The reported mean latency and throughput were calculated as arithmetic means across the 40 runs for each configuration.

The two configurations were benchmarked sequentially rather than in randomized or interleaved order.

4.5 Response Quality Evaluation

The response-quality evaluation reused the same 20 base prompts as the performance benchmark, rather than introducing an independently sampled prompt dataset. Each prompt was extended with the following instruction:

> *Answer clearly and concisely in no more than 100 words.*

One response per prompt and configuration was generated using greedy decoding and a maximum of 256 new tokens, producing 40 responses in total.

Instruction compliance was operationalized as whether the generated response contained no more than 100 whitespace-separated words, as counted using Python's `split()` method.

The experiment also recorded whether generation ended with a recognized end-of-sequence or end-of-turn token.

For exploratory response quality assessment, generated answers were evaluated on two dimensions: factual correctness and completeness.

The evaluation used two anonymization procedures. Responses for prompts 1–5 were scored using an initial identifier mapping, while responses for prompts 6–20 were scored using a revised

mapping in which responses were shuffled before anonymous identifiers were assigned.

The resulting scores were subsequently matched to their original prompt IDs and inference configurations using the corresponding mapping files.

The procedure was AI-assisted and did not involve independent human raters. Furthermore, the initial identifier mapping could reveal the inference configuration. Therefore, the evaluation must not be interpreted as fully blinded or independent.

Each dimension received a score from 0 to 2, producing a total score from 0 to 4 per response.

The scores were assigned through AI-assisted qualitative assessment rather than independent human evaluation. Consequently, the resulting quality scores are treated as exploratory indicators rather than validated measures of response correctness.

4.6 Paired Quality Analysis

Quality scores were compared across configurations using matched prompt IDs.

For each prompt, the score difference was defined as:

$$D_i = Q_{i,\mathrm{FP16}} - Q_{i,\mathrm{NF4}}. \tag{4}$$

The mean paired difference was calculated across the 20 prompts.

A percentile bootstrap procedure with 10,000 resamples of paired prompt differences was used to construct a 95% confidence interval for the mean difference. A fixed random seed of 42 was used for this analysis.

Given the small, purpose-built prompt set and subjective scoring procedure, the interval is interpreted cautiously and does not establish general equivalence between the configurations.

5 Results

This section presents the observed results for GPU memory allocation, inference performance, response quality, and instruction compliance.

5.1 GPU Memory Allocation

The FP16 configuration required an additional 0.9255 GiB of PyTorch-allocated GPU memory during model loading, compared with 0.4421 GiB for the 4-bit NF4 configuration.

This corresponds to a reduction of approximately 52.2% in measured allocation for NF4. The measured allocation difference is illustrated in Figure 1.

5.2 Inference Throughput and Latency

The main performance benchmark comprised 40 timed generations per configuration.

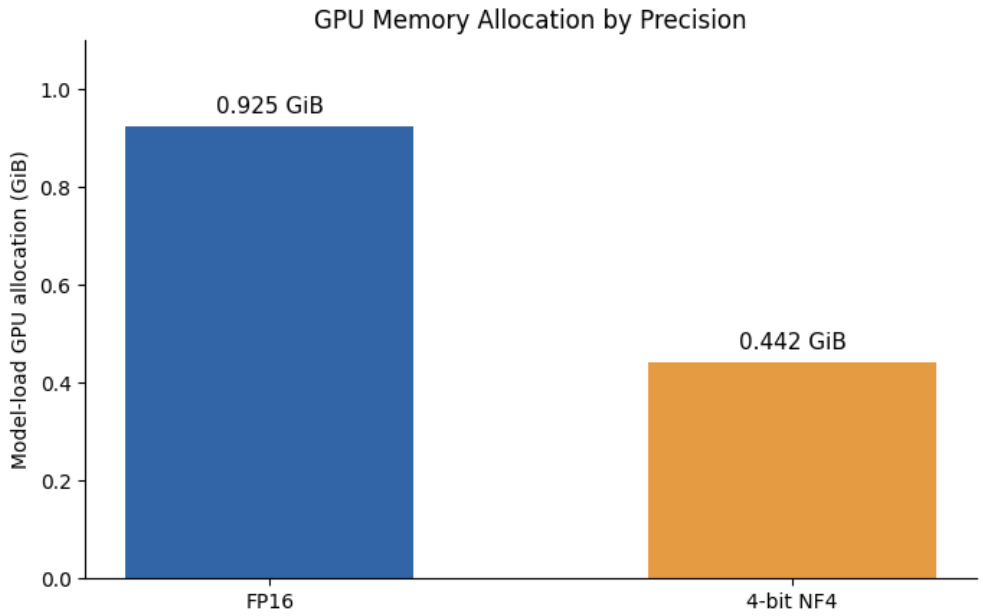


Figure 1: Change in PyTorch GPU memory allocation during model loading for FP16 and 4-bit NF4 configurations.

FP16 achieved a mean throughput of 29.753 generated tokens per second, compared with 19.936 tokens per second for 4-bit NF4.

Mean generation latency was 2.152 seconds for FP16 and 3.248 seconds for NF4.

The numerical results are summarized in Table 1, while Figure 2 visualizes the throughput and latency comparison.

Table 1: Mean inference performance across 40 generations per configuration.

Metric	FP16	4-bit NF4
Throughput (tokens/s)	29.753	19.936
Latency (s)	2.152	3.248

5.3 Exploratory Response Quality

The AI-assisted evaluation assigned mean total quality scores of 3.1 out of 4 for FP16 and 3.0 out of 4 for NF4.

Across the 20 paired prompts, FP16 received a higher score on five prompts, NF4 received a higher score on four prompts, and eleven prompts received equal scores.

The mean paired difference (FP16 minus NF4) was 0.100 points. A paired bootstrap analysis with 10,000 resamples produced a 95% confidence interval of [-0.350, 0.550].

The interval includes zero, and these exploratory results do not provide strong evidence that either configuration consistently produces higher-quality responses.

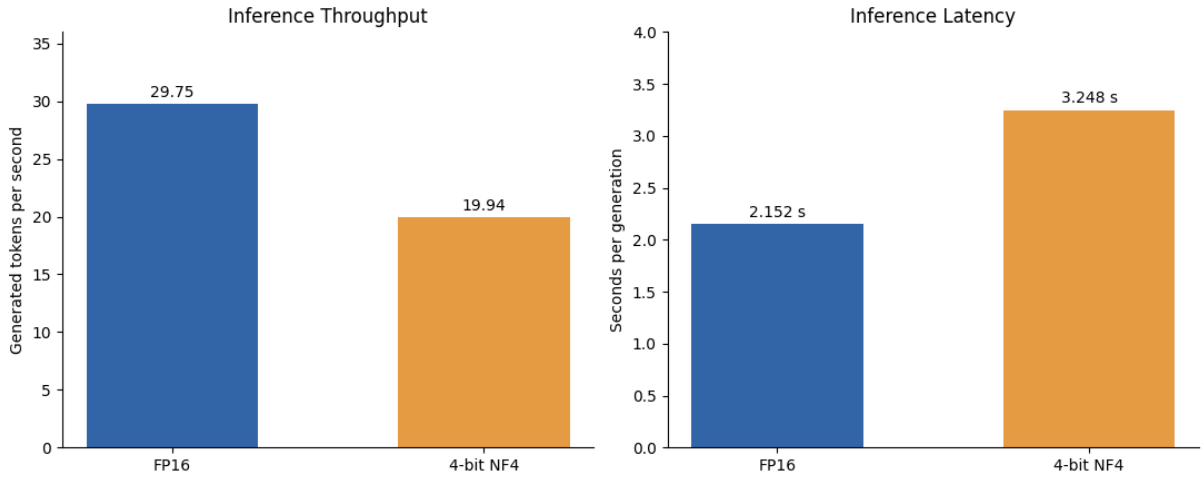


Figure 2: Mean generation throughput and latency for FP16 and 4-bit NF4.

5.4 Instruction Compliance

For the explicit 100-word response limit, 10 of 20 FP16 responses (50%) and 17 of 20 NF4 responses (85%) satisfied the constraint.

Additionally, 18 of 20 FP16 responses and all 20 NF4 responses terminated with a recognized end-of-sequence or end-of-turn token.

Figure 3 presents the exploratory quality scores alongside the observed instruction-compliance rates.

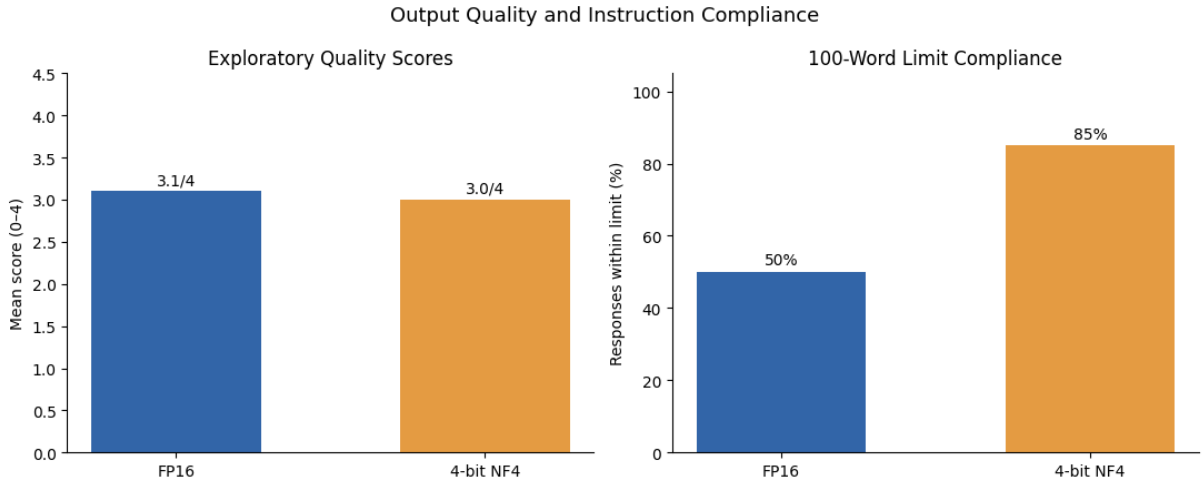


Figure 3: Exploratory mean quality scores and compliance with the 100-word response limit across 20 prompts.

The quality and instruction-compliance results describe the selected prompt set and should not be generalized to broader language-model workloads without additional evaluation.

6 Discussion

6.1 Memory Efficiency and Inference Speed

The results demonstrate an important distinction between memory efficiency and computational performance. Although 4-bit NF4 quantization reduced PyTorch-reported GPU memory allocation during model loading by approximately 52.2%, it did not improve generation speed in the main benchmark.

Instead, FP16 achieved approximately 49.2% higher mean generation throughput than NF4. The NF4 configuration also exhibited approximately 50.9% higher mean latency than FP16.

This outcome is consistent with the possibility that the overhead associated with quantized inference, including dequantization operations and implementation-specific kernel behavior, may outweigh potential computational benefits for this particular model and hardware configuration.

However, these mechanisms were not directly profiled in the present study. Therefore, they should be considered plausible explanations rather than experimentally established causes.

6.2 Resource-Aware Deployment Trade-offs

The findings suggest that selecting an inference configuration involves trade-offs rather than a single optimization objective.

For a deployment scenario where model-loading memory allocation is the primary constraint, 4-bit NF4 may be attractive because of its substantially smaller measured memory footprint.

Conversely, when generation throughput or response latency is the dominant requirement, FP16 performed better under the evaluated conditions.

These observations illustrate why deployment decisions should consider multiple operational metrics rather than assuming that reduced numerical precision necessarily produces faster inference.

The present experiment does not determine which configuration is optimal for a particular production application, since application-specific constraints and additional measurements would be required.

6.3 Response Quality and Reliability

The exploratory quality evaluation revealed only a small difference between the mean scores of FP16 and NF4.

The paired bootstrap confidence interval included zero, indicating that the observed difference does not provide strong evidence of consistent quality superiority for either configuration within the evaluation.

Nevertheless, the instruction-compliance experiment revealed a larger descriptive difference: NF4 satisfied the explicit 100-word response limit in 85% of cases, compared with 50% for FP16.

This observation suggests that numerical representation may influence generated response behavior, even under deterministic decoding.

However, the evaluation cannot establish that quantization generally improves instruction following. The result may depend on the specific prompts, model, generation settings, and word-count criterion used in this study.

Moreover, instruction compliance and factual correctness are distinct properties. A response satisfying a length constraint is not necessarily more accurate or more useful.

6.4 Implications for Sustainable AI

Resource-efficient model deployment is relevant to the broader objective of sustainable AI engineering.

Reducing the memory footprint of a model may enable deployment on devices with more limited memory capacity or support different resource-allocation strategies.

However, the present findings also show that memory savings can coexist with lower generation throughput and higher latency.

Consequently, memory reduction alone cannot be interpreted as evidence of lower energy consumption or reduced environmental impact.

A comprehensive sustainability assessment would require direct or appropriately validated measurements of energy usage, workload characteristics, hardware utilization, and potentially lifecycle impacts.

6.5 Relevance to Digital Sovereignty

The ability to execute language models within infrastructure controlled by an organization can be relevant to digital sovereignty, particularly where data governance, infrastructure independence, or service-provider dependencies are important.

Memory-efficient inference may help expand the range of hardware on which locally deployed models can operate.

Nevertheless, digital sovereignty is a broader organizational and technical concept that cannot be assessed through memory and inference benchmarks alone.

The present study does not measure data control, regulatory compliance, vendor dependence, or organizational autonomy. Its contribution in this area is therefore limited to identifying resource-related engineering trade-offs that could inform future investigations of independently managed AI deployment.

7 Threats to Validity and Limitations

This study has several limitations that should be considered when interpreting its findings.

7.1 Internal Validity

The FP16 and NF4 configurations were evaluated sequentially rather than using randomized or interleaved execution order.

Although warm-up generation and GPU synchronization were used in the performance benchmark, differences in runtime conditions, GPU clock behavior, thermal state, or system activity may have influenced the observed performance.

The study did not systematically monitor these potential confounding factors or repeat the complete benchmark across independent runtime sessions.

Consequently, the observed differences should not be interpreted as fully isolated causal effects of numerical precision alone.

7.2 External Validity

The experiments used one language model, Qwen2.5-0.5B-Instruct, and one GPU type, NVIDIA Tesla T4.

The benchmark also relied on a small, purpose-built collection of 20 English-language prompts focused on AI and software engineering topics.

These conditions do not represent the full diversity of language models, hardware architectures, prompt lengths, application domains, or production inference workloads.

The results therefore cannot be generalized directly to larger models, different GPUs, CPU-based inference, or edge devices.

7.3 Measurement Validity

The memory experiment measured the change in PyTorch-reported GPU allocation during model loading.

It did not measure peak GPU memory usage, total device memory consumption, or end-to-end deployment memory requirements.

Similarly, inference latency was measured for the generation call under the selected implementation. It does not include all possible production overheads, such as network communication, request queuing, or service orchestration.

Throughput and latency were calculated from the same generation runs and should not be interpreted as independent measurements of system behavior.

No direct measurements of electrical power, energy usage, or carbon emissions were made.

Therefore, the results do not establish environmental sustainability improvements.

7.4 Quality Evaluation Validity

Response quality was assessed using an exploratory AI-assisted scoring procedure rather than independent human annotators or a validated external evaluation benchmark.

Although anonymous identifiers were used, the assessment should not be characterized as a fully independent or fully blinded evaluation.

The scoring rubric considered factual correctness and completeness, but did not comprehensively assess hallucination risk, safety, robustness, or usefulness in real-world applications.

The 100-word compliance measurement used a simple whitespace-based word count, which may differ from linguistically informed tokenization or word-counting procedures.

The bootstrap confidence interval reflects uncertainty under resampling of the selected paired prompts. It does not account for all sources of evaluator bias, prompt-selection bias, or model-dependent variability.

7.5 Reproducibility Limitations

The study records the model identifier, software versions, hardware configuration, generation settings, and experimental output files.

However, the exact model repository revision was not pinned during the original experiments.

Moreover, execution within a managed cloud notebook environment may introduce variability across sessions.

Accordingly, the study provides documented experimental procedures and supporting artifacts, but exact numerical replication across different environments is not guaranteed.

8 Reproducibility and Research Artifacts

The experimental artifacts include a Google Colab notebook, prompt datasets, recorded benchmark results, quality assessment files, software environment information, and visualizations.

The notebook documents the experimental procedures, while the CSV files preserve the observed measurements and evaluation outputs. The accompanying environment record identifies the software versions and GPU configuration used in the study.

Together, these materials support inspection of the experimental procedures and independent attempts to reproduce the reported measurements.

The experiments were conducted using publicly constructed prompts and a publicly available language model. No private thesis data or proprietary datasets were used.

The artifacts are currently maintained as a research archive. Public release of a reviewed, sanitized version is planned separately.

As discussed in Section 7, exact numerical replication is not guaranteed across different hardware or runtime environments.

9 Conclusion and Future Work

This study presented an empirical comparison of FP16 and 4-bit NF4 inference for Qwen2.5-0.5B-Instruct on an NVIDIA Tesla T4 GPU, examining GPU memory allocation, generation

performance, exploratory response quality, and instruction compliance.

The results revealed a substantial trade-off between model-loading memory allocation and inference speed. The NF4 configuration reduced measured PyTorch GPU memory allocation by approximately 52.2% compared with FP16. However, FP16 achieved approximately 49.2% higher mean generation throughput and lower generation latency in the main benchmark.

The exploratory quality evaluation showed similar mean scores for both configurations, with no strong evidence of consistent superiority under the selected scoring procedure. In contrast, NF4 demonstrated higher compliance with an explicit 100-word response limit within the evaluated prompt set.

Taken together, these findings indicate that lower-precision model representation should not be treated as an unconditional inference optimization. Deployment choices require consideration of multiple criteria, including memory constraints, performance requirements, and output behavior.

Several directions for future research follow from these observations.

First, the evaluation could be extended to additional small language models, quantization methods, and hardware architectures to investigate the generality of the observed trade-offs.

Second, direct energy measurements and hardware utilization profiling could help clarify the relationship between memory efficiency, computational performance, and environmental sustainability.

Third, larger and more diverse evaluation datasets, together with independent human assessment, could provide stronger evidence regarding response quality and instruction compliance.

Finally, future work could investigate resource-aware inference selection frameworks that consider memory capacity, latency requirements, output quality, and organizational deployment constraints.

Such frameworks may contribute to the development of more efficient and independently managed AI systems, although their sustainability and digital-sovereignty benefits would require separate empirical validation.

References

[1] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[3] An Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.